

Self-Harness: Harnesses That Improve Themselves

Hangfan Zhang¹, Shao Zhang¹, Kangcong Li¹, Chen Zhang¹

Yang Chen¹, Yiqun Zhang¹, Lei Bai^{1,†}, Shuyue Hu^{1,†}

¹Shanghai Artificial Intelligence Laboratory, {zhanghangfan, zhangshao, hushuyue}@pjlab.org.cn

Abstract

The performance of LLM-based agents is jointly shaped by their base models and the harnesses that mediate their interaction with the environment. Because different models exhibit distinct behaviors, effective harness design is inherently model-specific. Yet agent harnesses are still largely engineered by human experts, a paradigm that scales poorly as modern LLMs become increasingly diverse and rapidly evolving. In this paper, we introduce *Self-Harness*, a new paradigm in which an LLM-based agent improves its own operating harness, without relying on human engineers or stronger external agents. We operationalize Self-Harness as an iterative loop with three stages: *Weakness Mining*, which identifies model-specific failure patterns from execution traces; *Harness Proposal*, which generates diverse yet minimal harness modifications tied to these failures; and *Proposal Validation*, which accepts candidate edits only after regression testing. We instantiate Self-Harness across Terminal-Bench-2.0, SWE-bench Verified, and AppWorld using a minimal initial harness and three base models from diverse families: MiniMax M2.5, Qwen3.5-35B-A3B, and GLM-5. Across all nine model-benchmark combinations, every final harness improves both held-in and held-out pass rates, with overall relative gains of up to 132%. Qualitative analyses further show that the retained mechanisms address benchmark-specific bottlenecks in artifact handling and runtime control, software-patch verification, and application-state retrieval. These results suggest a path toward LLM-based agents that are not merely shaped by their harnesses, but can also participate in reshaping them.

For a conscious being, to exist is to change, to change is to mature, to mature is to go on creating oneself endlessly.

—Henri Bergson, *Creative Evolution*

1. Introduction

To date, LLM-based agents are not shaped by their base model alone, but also by their *harness*: the surrounding system that situates the model and mediates its interaction with the environment. Although there is no universally accepted definition, a harness may include system prompts, tools, runtime mechanisms, verification rules, orchestration logic, and failure-recovery procedures. The same base model can thus exhibit substantially different performance under different harnesses (Lee et al., 2026; Lin et al., 2026; Yang et al., 2024).

From early frameworks such as ReAct (Yao et al., 2023) to product- and platform-level systems such as Claude Code, Codex, and OpenHands, harnesses have largely been engineered by human experts (Liu et al., 2026; OpenAI, 2026; Wang et al., 2025; Zhang et al., 2025c; Zhu et al., 2026).

[†] Corresponding authors

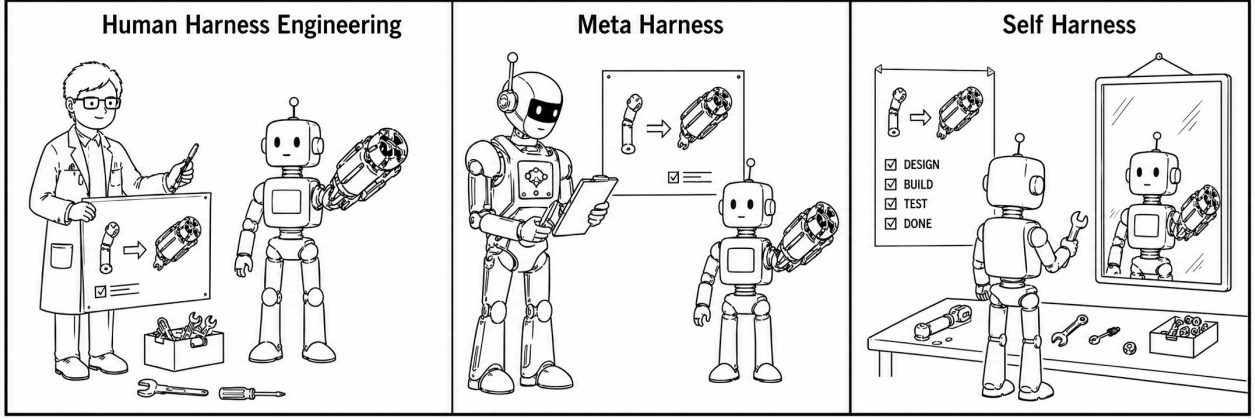


Figure 1 | Three paradigms of harness improvement. In human harness engineering, human engineers manually revise the agent harness. In Meta-Harness, a stronger external agent guides the improvement of a weaker target agent. In Self-Harness, the agent improves its own operating harness.

While effective, this human-centered paradigm does not scale well with the diversity and rapid evolution of modern LLMs. Different models can exhibit distinct behavioral patterns, tool-use habits, error modes, and sensitivities to prompting (Qin et al., 2023; Schulhoff et al., 2025; Sclar et al., 2024); consequently, a harness that works well for one model may be suboptimal for another (Lee et al., 2026; Lin et al., 2026; Sclar et al., 2024). As new models continue to be released at a rapid pace, manually redesigning and tuning a model-specific harness for each model becomes increasingly costly and untenable.

In this paper, we explore a novel paradigm, *Self-Harness*: enabling an LLM-based agent to improve the very harness through which it operates (Figure 1). Unlike recent approaches that use stronger external agents to improve the harnesses of weaker ones (Lee et al., 2026; Lin et al., 2026), Self-Harness seeks to internalize this improvement loop within the target agent itself. This paradigm reduces dependence on external guidance that may be costly, unavailable for frontier models, or mismatched to the target model’s failure modes. More broadly, in Bergson’s terms, this points toward a technical analogue of self-creation: a system not merely changed from without, but continually “going on creating itself.”

We operationalize Self-Harness as an improvement loop that repeatedly turns behavioral evidence into harness updates (Figure 2). The loop consists of three stages. **Weakness Mining**: Starting from an initial harness, the agent with a fixed model is run on a set of tasks, producing execution traces with verifiable outcomes. The agent then clusters failed traces, allowing it to reason about model-specific failure patterns rather than isolated mistakes. **Harness Proposal**: Based on these failure patterns, the agent generates a small set of diverse yet minimal harness modifications, each tied to a specific failure mechanism. This constraint ensures that proposed edits remain targeted rather than overly general. **Proposal Validation**: Candidate modifications are evaluated through regression tests, and an edit is promoted only if it improves performance without causing measurable degradation on held-out tasks. If multiple candidate modifications pass the regression tests, they are merged into the next version of the harness, which then serves as the starting point for the next iteration.

In our experiments, we instantiate Self-Harness with a minimal initial harness (Figure 3) and three base models from diverse families: MiniMax M2.5, Qwen3.5-35B-A3B, and GLM-5 (GLM-5 Team, 2026; MiniMax, 2026; Qwen Team, 2026), across Terminal-Bench-2.0, SWE-bench Verified, and AppWorld. Across the nine model–benchmark combinations, every final harness improves both held-in and held-out pass rates (Table 1). On Terminal-Bench-2.0, Self-Harness consistently

improves performance across all three models. For held-in tasks, which provide execution traces to the evaluation system, the pass rate is increased from 43.0% to 50.0% for MiniMax M2.5, from 15.1% to 36.0% for Qwen3.5-35B-A3B, and from 47.7% to 57.0% for GLM-5. For held-out tasks, whose execution traces are never used as inputs to the evaluation system, the improvements remain substantial. The pass rate is increased from 40.5% to 61.9% for MiniMax M2.5, from 23.8% to 38.1% for Qwen3.5-35B-A3B, and from 42.9% to 57.1% for GLM-5. The same pattern holds on SWE-bench Verified and AppWorld: every model improves on both splits, with overall Pass gains of 3.5–22.0 and 10.3–40.6 percentage points, respectively. These results indicate that Self-Harness can evolve an initial harness into model-specific ones better suited to different base models. Moreover, it can discover broadly useful harness modifications that transfer to tasks whose traces are hidden from the proposer.

Qualitative analyses further show that Self-Harness does more than simply make the prompt longer or add generic instructions. Instead, it introduces targeted changes that reflect the recurring problems each model encounters during execution, turning model-specific weaknesses into concrete harness-level interventions. For MiniMax M2.5, the changes encourage the agent to create required output files earlier, handle structured tool outputs more carefully, and stop unproductive tool-use loops before they become too long. For Qwen3.5-35B-A3B, the changes focus on checking dependencies in advance, avoiding repeated failed commands, breaking cycles of endless exploration, and reminding the agent to produce the required artifacts after tool errors. For GLM-5, the changes mainly help the agent preserve environment settings across shell commands and move more quickly from exploration to implementation and testing. Notably, Self-Harness can also introduce broader structural mechanisms, such as subagent-based decomposition and middleware creation, that go beyond local failure repair and improve the overall organization of problem solving.

To summarize, our key contributions are as follows:

- We propose Self-Harness, a novel paradigm for harness improvement that enables an LLM-based agent to design and refine the harness through which it operates, tailoring it to its own base model without human engineering effort or guidance from a stronger external agent.
- We operationalize Self-Harness as an iterative loop that turns each model’s behavioral evidence into model-specific harness updates: it evaluates execution traces to identify recurring failure patterns, generates diverse yet minimal candidate edits, and promotes only those that pass regression tests.
- Experiments show that Self-Harness improves performance across all nine model–benchmark combinations, with overall Pass gains of up to 40.6 percentage points and relative improvements of up to 132%; qualitative analyses further confirm that different models benefit from distinct harness changes, suggesting that Self-Harness can turn model-specific weaknesses into concrete harness changes.

2. Background and Related Work

From prompts to agent harnesses. Prompt engineering and context engineering show that fixed models can be steered by instructions, demonstrations, retrieved evidence, memory, tool state, and dynamically constructed inputs (Lewis et al., 2020; Liang et al., 2026; Liu et al., 2021; Mei et al., 2025; Packer et al., 2024; Schulhoff et al., 2025; Wei et al., 2022; Xu et al., 2026). Agentic systems extend this control surface from a single input to an execution environment: the model acts, observes consequences, uses tools, receives feedback, and follows runtime policies. ReAct, SWE-agent, Claude Code, and SemaClaw/OpenClaw illustrate how such surrounding mechanisms shape long-horizon agent behavior and software-engineering performance (Liu et al., 2026; Yang et al., 2024; Yao et al.,

2023; Zhu et al., 2026).

We use *harness* for this surrounding system layer: prompts, tools, memory, verification rules, permission policies, adapters, and runtime mechanisms that mediate between the model and the environment. Many important agent failures are failures of this layer rather than failures of an isolated model response: an agent may report success without checking an artifact, retry an unproductive action pattern, lose the source of truth in a long context, or lack a recovery action. These behaviors emerge from the interaction between instructions, observations, tools, and runtime control, so improving them requires changing more than prompt text.

Self-improving agents and automated agent design. A growing line of work studies systems that adapt their inputs, memories, contexts, or workflows over time (Shinn et al., 2023; Zelikman et al., 2024; Zhang et al., 2025a, 2026). Reflexion stores verbal feedback for later attempts (Shinn et al., 2023), agentic context engineering evolves contexts for later model calls (Zhang et al., 2026), and STOP studies recursive self-improvement for code generation (Zelikman et al., 2024). These methods show that fixed models can benefit from accumulated feedback, but the adapted object is usually a response strategy, memory, context, or generated program rather than a declared agent harness state.

A second line optimizes agent designs from outside the evaluated agent. Automated Design of Agentic Systems searches over agent designs, language agents can be represented as optimizable graphs, and Meta-Harness directly optimizes harness code using source code, scores, and traces from prior candidates (Hu et al., 2025; Lee et al., 2026; Zhuge et al., 2024). These systems motivate harness-level optimization, but they frame improvement as an external search or optimization process rather than as a bounded edit proposed by the evaluated model under its current harness.

Finally, scientific discovery and self-evolving agent systems such as The AI Scientist, AI Scientist-v2, AlphaEvolve, Alita, Godel Agent, and Darwin Godel Machine automate broader loops of research, algorithm design, or capability expansion (Fu et al., 2026; Lu et al., 2024; Novikov et al., 2025; Qiu et al., 2025; Yamada et al., 2025; Yin et al., 2025; Zhang et al., 2025b). Self-Harness is closest in spirit to this self-improvement literature and to automated harness optimization, but it studies a narrower controlled setting: whether the same fixed model, operating under the current harness, can propose a bounded candidate change to the harness that governs its own future behavior.

3. Self-Harness: An Iterative Loop for Model-Specific Harness Improvement

Human harness engineering improves agent harnesses through expert inspection and manual revision, while external optimizer approaches treat harness design choices as a searchable space. Self-Harness studies a middle ground in which a fixed model iteratively improves the harness around itself through an explicit self-improvement loop driven by execution evidence. In each iteration, the evaluation system runs the current harness and mines recurring failure patterns from clustered execution traces to produce structured evidence. Given this evidence, the same model is invoked in a proposer role to generate a set of diverse yet minimal candidate harness modifications, each targeting a specific failure mechanism without replacing the overall control architecture. Candidate edits are then validated through regression testing on held-out tasks, and an explicit acceptance rule promotes only those edits that improve performance without introducing unacceptable regressions.

3.1. Preliminary

We use *harness* to denote the non-parametric scaffolding that governs how a fixed language model is deployed as an agent. A harness includes the instructions, the available tools, memory and state-

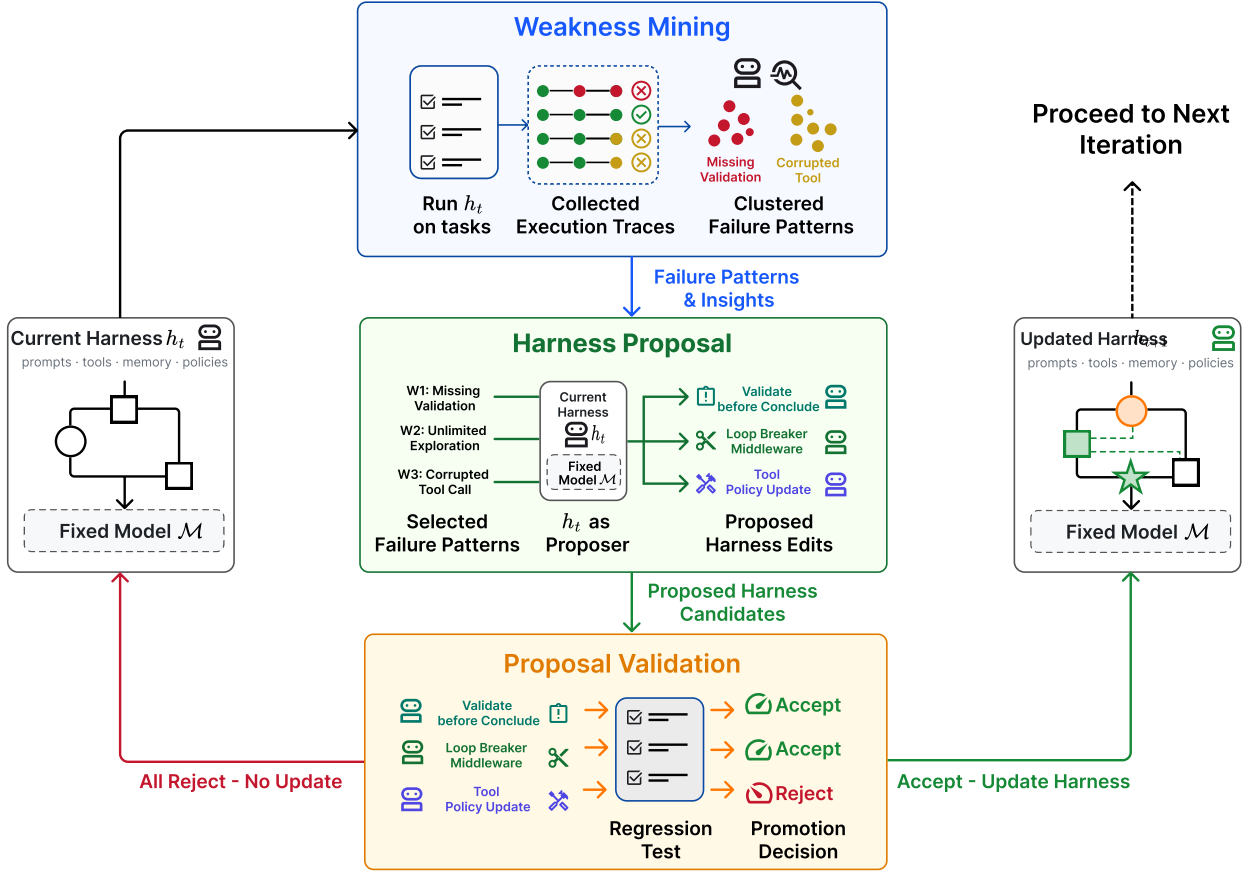


Figure 2 | Overview of one Self-Harness optimization loop. The current harness h_t with fixed model is evaluated on tasks to collect execution traces, which are clustered into verifier-grounded failure patterns. The same model is then invoked under the current harness as a proposer, using the mined failure patterns to generate bounded candidate harness edits. Candidate edits are evaluated by regression tests on held-in and held-out splits. Accepted candidates are merged to update the harness to h_{t+1} , while rejected candidates are logged without changing the active harness. Throughout the loop, the model weights and evaluator remain fixed; only the surrounding harness is modified.

management mechanisms, etc. The harness does not modify the model parameters; instead, it specifies the execution protocol through which the model observes a task, takes actions, invokes tools, checks intermediate artifacts, and produces a final answer.

Formally, let M be a fixed language model and let h denote an agent harness. Given a task instance x , running M under harness h produces an execution trace τ and an output y . The trace records the messages, tool calls, and verifier outcomes. An evaluator then maps the task, trace, and output to a behavioral outcome, such as pass/fail. In this work, the model M and evaluator $\mathcal{E}$ are held fixed, while the harness is treated as the object of improvement. Self-Harness therefore operates over a lineage of harnesses $h_0, h_1, \dots$, where each transition corresponds to a bounded edit to the execution protocol rather than an update to the model weights.

3.2. Weakness Mining: Identifying Failure Patterns from Clustered Execution Traces

The first stage of Self-Harness converts behavioral failures into structured evidence for harness revision. At round t , we run the fixed model M under the current harness h_t on a held-in split D_{in} . For each task instance $x_i \in D_{\text{in}}$, the run produces an output y_i and an execution trace τ_i . The evaluator $\mathcal{E}$ then

Algorithm 1 Self-Harness

Require: fixed model M , initial harness h_0 , held-in split D_{in} , held-out split D_{ho} , evaluator $\mathcal{E}$, proposal width K , rounds T

Ensure: final harness h_T

```

1: for  $t = 0, 1, \dots, T - 1$  do
2:    $(P_{\text{in}}(h_t), P_{\text{ho}}(h_t), R_t) \leftarrow \text{EVALUATE}(M, h_t, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$ 
3:    $B_t \leftarrow \text{BUILDEVIDENCEBUNDLE}(R_t)$  ▷ from held-in verifier-grounded failures
4:    $\mathcal{P}_t \leftarrow \text{PARALLELPROPOSE}(M, h_t, B_t, K)$  ▷  $\mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K$ 
5:    $\mathcal{A}_t \leftarrow \emptyset$ 
6:   for all  $(\Delta_j, a_j) \in \mathcal{P}_t$  do
7:      $h_t^{(j)} \leftarrow \Delta_j(h_t)$ 
8:      $(P_{\text{in}}(h_t^{(j)}), P_{\text{ho}}(h_t^{(j)}), R_t^{(j)}) \leftarrow \text{EVALUATE}(M, h_t^{(j)}, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$ 
9:      $\Delta_{\text{in}}^{(j)} \leftarrow P_{\text{in}}(h_t^{(j)}) - P_{\text{in}}(h_t)$ 
10:     $\Delta_{\text{ho}}^{(j)} \leftarrow P_{\text{ho}}(h_t^{(j)}) - P_{\text{ho}}(h_t)$ 
11:    if  $\Delta_{\text{in}}^{(j)} \geq 0$  and  $\Delta_{\text{ho}}^{(j)} \geq 0$  and  $\max(\Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)}) > 0$  then
12:       $\mathcal{A}_t \leftarrow \mathcal{A}_t \cup \{(h_t^{(j)}, \Delta_j, a_j, \Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)})\}$ 
13:       $\text{ACCEPT}(\Delta_j)$  ▷ passed acceptance rule
14:    else
15:       $\text{REJECT}(\Delta_j)$ 
16:    end if
17:  end for
18:  if  $\mathcal{A}_t = \emptyset$  then
19:     $h_{t+1} \leftarrow h_t$  ▷ no accepted candidate
20:  else
21:     $h_{t+1} \leftarrow \text{MERGEACCEPTED}(h_t, \mathcal{A}_t)$  ▷ accepted edits are merged
22:  end if
23: end for
24: return  $h_T$ 

```

assigns an outcome $z_i = \mathcal{E}(x_i, \tau_i, y_i)$, such as pass or fail. This yields a trace record

$$r_i = (x_i, \tau_i, y_i, z_i),$$

and a round-level record set $R_t = \{r_i\}_{i=1}^{|D_{\text{in}}|}$. Since both M and $\mathcal{E}$ are fixed, changes in these records across rounds can be attributed to changes in the harness.

A central role of the evaluation system is to avoid treating failures as isolated anecdotes. We therefore focus on the subset of failed records

$$F_t = \{r_i \in R_t \mid z_i = \text{fail}\}.$$

and cluster them by verifier-grounded failure signatures. For each failed record r_i , the evaluation system analyzes the trace as evidence for why the evaluator rejected the run. It identifies the terminal failure reason exposed by the verifier, the agent-side behavior connected to that terminal failure, and the causal status of that behavior within the trace. This attribution step prevents the clustering procedure from conflating superficial symptoms with reusable failure mechanisms: two runs may share the same verifier outcome, such as a timeout or missing artifact, while requiring different harness changes because the underlying agent behaviors differ.

We write this attribution as a failure signature

$$\phi(r_i) = (c_i, q_i, m_i),$$

where c_i denotes the terminal verifier-level cause, q_i denotes the causal status of the relevant agent behavior, and m_i denotes the abstract agent mechanism exposed by the trace. Failures are clustered by exact agreement of this signature:

$$C_\phi = \{r_i \in F_t \mid \phi(r_i) = \phi\}$$

Thus, the clustering is deterministic and evaluator-grounded: two failed cases are grouped together only when they agree on what the verifier ultimately rejected, how the agent behavior contributed to that rejection, and which reusable behavioral mechanism was involved. The goal is not to discover latent semantic similarity among traces, but to aggregate failures that plausibly admit the same harness-level intervention.

For each cluster C_ϕ , the evaluation system constructs a structured failure pattern containing its cluster size, representative task instances, shared trace symptoms, verifier evidence, and the inferred agent mechanism. Clusters are then ordered by their support and estimated actionability, so that the proposer is exposed first to recurring mechanisms that are more likely to map to a high-value harness modification.

The output of this stage is an evidence bundle B_t summarizing the dominant failure patterns observed under h_t . Importantly, B_t does not prescribe a harness edit. It separates verifier-level failure from agent-level mechanism, allowing the proposer to target a specific reusable weakness rather than patching a coarse outcome such as timeout, assertion failure, or missing output. This keeps the evaluator distinct from the optimizer while ensuring that subsequent candidate modifications are grounded in explicit cross-case evidence.

3.3. Harness Proposal: Exploring Diverse yet Minimal Candidate Modifications

Given the evidence bundle B_t , the proposal stage translates recurring failure patterns into candidate harness edits. The proposer is not an external optimizer with unrestricted access to the search space. Instead, we invoke the same fixed model M with current harness h_t in a proposer role and provide it with a bounded proposal context: the editable surfaces of the current harness, the verifier-grounded failure patterns from the evaluation system, records of passing behaviors that should be preserved, and summaries of previously attempted edits. This context exposes the proposer to structured cross-case evidence rather than raw execution logs, encouraging it to reason about reusable failure mechanisms rather than individual task failures.

Self-Harness uses parallel proposal generation to explore several candidate improvements from the same evidence. The proposer generates K mutually distinct proposal bundles,

$$\mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K,$$

where each edit Δ_j maps the current harness to a candidate harness

$$h_t^{(j)} = \Delta_j(h_t).$$

and a_j is an audit record describing the targeted failure pattern, the edited harness surface, the expected behavioral effect, and the regression risks. Each proposal must be grounded in a primary failure mechanism and mapped to a concrete editable surface. The candidates are required to be materially distinct: they should not merely restate the same cluster, surface, or mechanism with

different wording. This parallel proposal step broadens exploration while keeping each candidate branch individually interpretable.

The proposer first selects target failure patterns from B_t . A pattern is considered a suitable target only if it is both supported by evidence and plausibly addressable by an editable harness surface. This addressability criterion is important because not every failure cluster implies a useful harness modification: some clusters reflect task-specific difficulty, unstable outcomes, or model capability limits rather than a missing execution rule. When multiple clusters are plausible, the proposer favors mechanisms that are concrete, recurrent, and likely to be mitigated by a narrow change to the execution protocol; weakly supported or non-addressable patterns are excluded rather than forced into a patch.

Diversity is encouraged across proposal branches, while minimality is enforced within each branch. A proposal may target a different failure mechanism, choose a different harness surface, or express a different hypothesis about how to improve execution. However, each individual edit is constrained to modify only the surface needed to address its selected mechanism, preserve unrelated harness behavior, and avoid broad rewrites of the agent control architecture.

3.4. Proposal Validation: Ensuring Robust Improvement through Regression Testing

A candidate harness edit is not adopted immediately after it is proposed. Instead, each candidate branch is treated as a new harness variant and evaluated under the same evaluator used to diagnose the current harness. For a proposal Δ_j , let $h_t^{(j)} = \Delta_j(h_t)$ denote the resulting candidate harness. We evaluate both the current harness h_t and the candidate harness $h_t^{(j)}$ on the held-in split D_{in} and the held-out split D_{ho} . The held-in split measures whether the proposal addresses the evidence that motivated it, while the held-out split serves as a regression test for behaviors that were not visible to the proposer.

Let $P_{\text{in}}(h)$ and $P_{\text{ho}}(h)$ denote the number of passed tasks for harness h on D_{in} and D_{ho} , respectively. We define the split-wise improvements of candidate $h_t^{(j)}$ over the current harness as

$$\Delta_{\text{in}}^{(j)} = P_{\text{in}}(h_t^{(j)}) - P_{\text{in}}(h_t),$$

and

$$\Delta_{\text{ho}}^{(j)} = P_{\text{ho}}(h_t^{(j)}) - P_{\text{ho}}(h_t).$$

A candidate is accepted only if it improves at least one split without degrading the other:

$$\Delta_{\text{in}}^{(j)} \geq 0, \quad \Delta_{\text{ho}}^{(j)} \geq 0, \quad \max(\Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)}) > 0.$$

This rule implements a conservative promotion criterion. Proposals that only trade off one split against the other are rejected, even if their total pass count increases. When evaluation is stochastic, we repeat candidate evaluation and apply the same rule to aggregate pass counts across repeats. This reduces the chance that a harness edit is promoted due to a single favorable run. If multiple compatible candidates satisfy the rule in the same round, their edits are merged into the next harness; rejected candidates remain logged but do not change the active harness. In addition to the pass-count rule, validation rejects proposals that do not modify any editable surface or fail execution before a valid evaluation result is obtained. For each evaluated candidate, the system records the changed surfaces, split-wise outcomes, evaluation repeats, proposal summary, and accept/reject decision, making each transition in the harness lineage auditable.

4. Experiments

We evaluate whether Self-Harness can improve agent performance by modifying only the harness around a fixed language model across three agentic benchmarks: Terminal-Bench-2.0, SWE-bench Verified, and AppWorld. Together, they cover terminal interaction in containerized environments, repository-level software repair, and multi-application workflows. Across three model backends, we start from a minimal benchmark-specific harness and let Self-Harness propose, validate, and promote bounded edits using held-in execution evidence and held-out regression gates.

4.1. Setup

Benchmarks. We evaluate Self-Harness on Terminal-Bench-2.0 (Merrill et al., 2026), SWE-bench Verified (Jimenez et al., 2024), and AppWorld (Trivedi et al., 2024). Terminal-Bench-2.0 contains 89 containerized terminal tasks that test general tool-based execution, including artifact management, command use, verification behavior, and recovery from execution errors. We evaluate on a fixed 64-case subset, excluding tasks that depend on unstable external web resources or require multimodal inputs. This filtering reduces evaluation noise from factors outside the harness. In particular, multimodal tasks require modality-specific input handling that is not exposed by our minimal initial harness; including them would primarily measure unsupported harness functionality rather than the effect of Self-Harness edits. SWE-bench Verified evaluates repository-level software repair: an agent must inspect a real codebase, implement a patch for a reported issue, and satisfy repository tests in the benchmark-provided per-instance execution environment. We use a fixed 100-case subset partitioned into 67 held-in and 33 held-out cases. Cases are sampled proportionally by repository. AppWorld evaluates multi-application workflows in which an agent interacts with application APIs and is graded by state-based unit tests. We use 180 examples divided evenly between held-in and held-out splits. The held-in split contains the official training tasks, while the held-out split is fixed by proportionally sampling from the official normal and challenge test partitions.

Models. We evaluate Self-Harness with three models: MiniMax M2.5 (MiniMax, 2026), Qwen3.5-35B-A3B (Qwen Team, 2026), and GLM-5 (GLM-5 Team, 2026). The model is held fixed across all harness variants and is also used in the proposal stage to generate edits from evaluator feedback. All comparisons are therefore within-model comparisons: the decoding configuration, budget, tool set, benchmark environment, and evaluator are kept unchanged while only the harness is allowed to vary. This isolates the effect of Self-Harness from changes in model capability or evaluation protocol.

Harness. The initial harness builds upon the DeepAgent (LangChain, 2026) SDK but is intentionally kept minimal: a short benchmark-facing system prompt, and the default filesystem and shell tools. Self-Harness can only change the harness definition file that configures how DeepAgent is instantiated and controlled to build a new harness candidate $h_t^{(j)}$. The editable surfaces correspond to declared configuration points in this harness, such as instruction, tools, verification guidance, etc. Figure 3 shows the initial harness implementation.

Splits and protocol. We fix the evaluated task set and partition it into a held-in split and a held-out split before running Self-Harness. The held-in split supplies the trajectories, verifier outcomes, and failure evidence exposed to the proposer, while the held-out split is never shown to the proposer and is used only by the automatic promotion gate. A candidate harness is promoted with the acceptance rule defined in Section 3.4. Task split assignments are fixed across harness variants, and each task

```

1 def build_system_prompt() -> str:
2     return """
3     You are running inside a Terminal Bench 2 Harbor task environment.
4
5     Use the built-in filesystem and shell tools to inspect the workspace, make
6     concrete edits, and verify outcomes against the actual task environment.
7
8     Do not assume synthetic datasets, domain-specific tools, or hidden fixtures
9     unless you discover them in the repo or runtime.
10    """.strip()
11
12    BASELINE_SYSTEM_PROMPT = build_system_prompt()
13
14
15
16    def build_memory_sources() -> list[str]:
17        return ["/AGENTS.md"]
18
19
20    def build_subagents() -> list[dict[str, Any]]:
21        return []
22
23
24    def build_skills() -> list[str]:
25        return []
26
27
28    def build_bootstrap_instruction() -> str:
29        return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
30
31
32    def build_execution_instruction() -> str:
33        return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the
34        task."
35
36
37    def build_verification_instruction() -> str:
38        return "Before concluding, verify the result with the most targeted command, file read, or test
39        you can run."
40
41
42    def build_failure_recovery_instruction() -> str:
43        return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
44
45
46    def build_runtime_control_policy() -> dict[str, Any]:
47        return {
48            "enabled": False,
49            "max_recent_tool_errors": None,
50            "max_total_tool_messages": None,
51            "instruction": None,
52        }
53
54
55    def build_fixed_harness_agent(
56        model: "BaseChatModel | str",
57        *,
58        backend: Any | None = None,
59    ):
60        if backend is not None:
61            return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT, backend=backend)
62        return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT)

```

Figure 3 | Initial harness and editable interface used as the starting point for Self-Harness. The harness is intentionally kept minimal, consisting only of the Terminal-Bench-2.0 default system prompt, the default DeepAgent tools (basic file reading, file writing, file editing, and shell execution), and the declared interfaces that Self-Harness is allowed to modify.

Model	Initial harness			Self-Harness		
	Held-in	Held-out	Overall	Held-in	Held-out	Overall
Terminal-Bench-2.0						
MiniMax M2.5	43.0	40.5	42.2	50.0 (+16%)	61.9 (+53%)	53.9 (+28%)
Qwen3.5-35B-A3B	15.1	23.8	18.0	36.0 (+138%)	38.1 (+60%)	36.7 (+104%)
GLM-5	47.7	42.9	46.1	57.0 (+20%)	57.1 (+33%)	57.0 (+24%)
SWE-bench Verified						
MiniMax M2.5	51.5	34.8	46.0	58.2 (+13%)	40.9 (+18%)	52.5 (+14%)
Qwen3.5-35B-A3B	20.1	18.2	19.5	42.5 (+111%)	39.4 (+116%)	41.5 (+113%)
GLM-5	53.7	48.5	52.0	58.2 (+8%)	50.0 (+3%)	55.5 (+7%)
AppWorld						
MiniMax M2.5	51.7	45.6	48.6	62.8 (+21%)	55.0 (+21%)	58.9 (+21%)
Qwen3.5-35B-A3B	25.0	20.0	22.5	60.0 (+140%)	44.4 (+122%)	52.2 (+132%)
GLM-5	47.8	41.1	44.4	92.2 (+93%)	77.8 (+89%)	85.0 (+91%)

Table 1 | Held-in, held-out, and overall pass rates (%) for the initial and final harnesses. Bold black values denote the final harness produced by Self-Harness.

starts from a fresh benchmark environment. These controls ensure that measured improvements come from harness changes.

Metrics. Our primary metric is *Pass (%)*, the percentage of evaluated task attempts that pass the corresponding benchmark’s official verifier, computed over two repeated attempts for each harness candidate unless otherwise specified. This measures mean single-attempt task success under a fixed harness configuration and evaluation protocol.

4.2. Main Results

Table 1 shows that every final harness improves Pass on both held-in and held-out splits across all nine model–benchmark combinations. The largest relative gain is 132% for Qwen3.5-35B-A3B on AppWorld, while the largest absolute gain is 40.6 percentage points for GLM-5 on AppWorld, from 44.4% to 85.0%. Meanwhile, the relative held-out gain exceeds the held-in gain in four combinations: MiniMax M2.5 and GLM-5 on Terminal-Bench-2.0, and MiniMax M2.5 and Qwen3.5-35B-A3B on SWE-bench Verified.

These results show that harness-level edits can yield measurable improvements while keeping the model backend, tool set, budget, benchmark environment, and evaluator fixed. The gains are not confined to the held-in failures used to construct proposal evidence: all three backends improve on the held-out split, and no promoted harness degrades either split. This supports the central design goal of Self-Harness: proposed edits should target reusable execution mechanisms rather than case-specific failures, and the regression gate should prevent improvements on one split from being promoted at the cost of another.

4.3. Experimental Analysis

Harness evolution and retained edits. Figure 4 presents four representative evolution trajectories spanning all three benchmarks, while Figures 5 and 6 show the retained code-level edits for the two Terminal-Bench-2.0 examples. On Terminal-Bench-2.0, the retained edits primarily target artifact handling, failure recovery, and runtime control; on SWE-bench Verified, they target patch validation

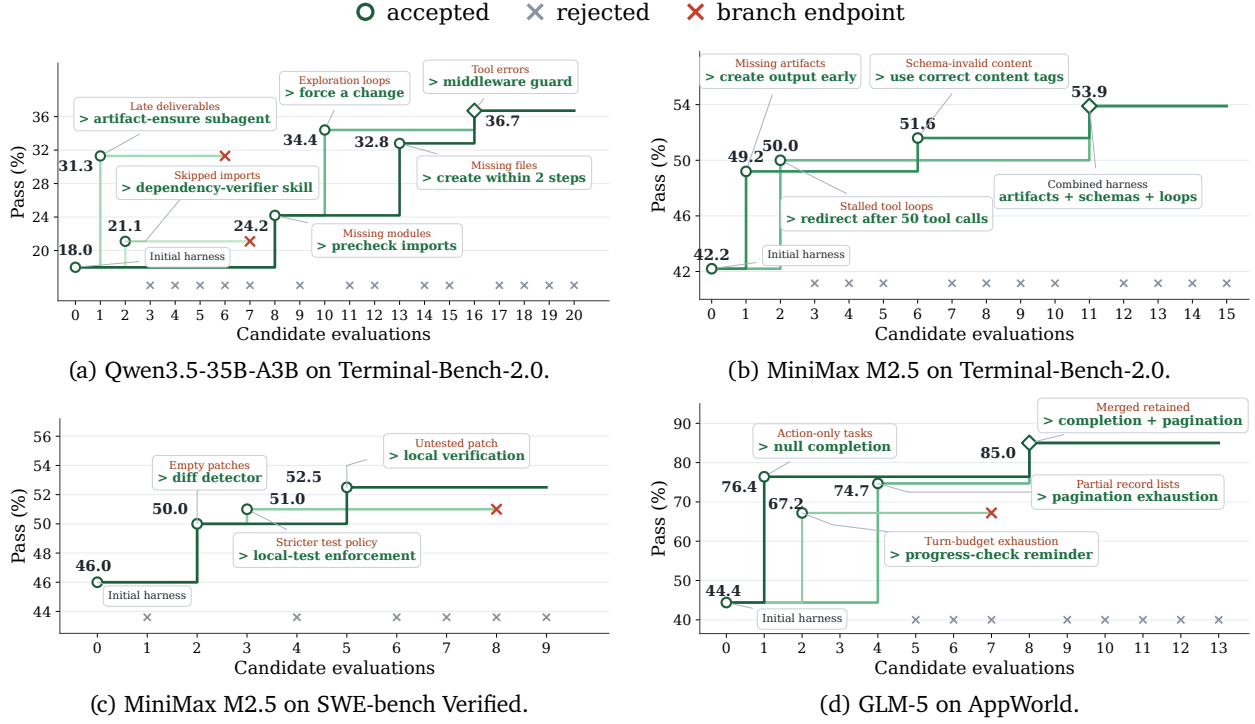


Figure 4 | Representative Self-Harness evolution trajectories across model backends and benchmarks. Green markers denote accepted harness candidates, gray crosses denote rejected candidates, and red crosses mark explored branch endpoints. Step lines connect candidates to their parent harness; the x-axis follows candidate-evaluation order.

and executable verification; and on AppWorld, complete state retrieval and correct completion semantics. The concrete implementation of these mechanisms remains model-specific. Across these runs, proposed edits passing validation are denoted as green nodes. Red nodes and grey slots denote rejected branches and failing trials, respectively. The corresponding GLM-5 Terminal-Bench-2.0 run is introduced in Appendix Figure 10, and Sections B.2 and B.3 report the remaining SWE-bench Verified and AppWorld evolution, code-level, and trace analyses.

For MiniMax M2.5, the harness improves from 42.2% to 53.9% pass rate. The retained edits in Figure 5 address missing required artifacts, schema-invalid tool content, and stalled tool-use loops, yielding a harness that creates required outputs earlier, handles structured tool content more carefully, and redirects execution after prolonged tool interaction.

The Qwen3.5 evolution run shown in Figure 4(a) starts at 18.0% pass rate and reaches 36.7% after merging the edits shown in Figure 6, which emphasize artifact checking, missing-artifact recovery, retry discipline, and tool-error-triggered middleware. These changes mainly improve the agent’s ability to recover from file-editing or tool failures and still leave verifier-required artifacts in place.

For GLM-5, the harness improves from 46.1% to 57.0% through edits targeting late artifacts, external computation, session-scoped tools, and implementation-oriented exploration. These edits make environment changes persist across shell commands and encourage the agent to move from prolonged exploration toward implementation and testing.

MiniMax M2.5: create output early

```

1 1 def build_bootstrap_instruction() -> str:
2 2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 3 + return (
4 4 +     "Start by inspecting the workspace and identifying the task's required output artifact. "
5 5 +     "Create an initial version of that artifact as early as possible, then iterate to improve it."
6 6 + )

```

MiniMax M2.5: use correct content tags

```

1 1 def build_execution_instruction() -> str:
2 2 - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3 3 + return (
4 4 +     "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task. "
5 5 +     "When using tools that return structured content, use the correct content type formats, such as 'image_url' "
6 6 +     "rather than 'image' for image references."
7 7 + )

```

MiniMax M2.5: redirect after 50 tool calls

```

1 1 def build_runtime_control_policy() -> dict[str, Any]:
2 2     return {
3 3 -     "enabled": False,
4 4 +     "enabled": True,
5 5 -     "max_recent_tool_errors": None,
6 6 +     "max_recent_tool_errors": 5,
7 7 -     "max_total_tool_messages": None,
8 8 +     "max_total_tool_messages": 50,
9 9 -     "instruction": None,
10 10 +     "instruction": (
11 11 +         "You appear to be stuck in a tool loop. Stop repeating the same failed path, summarize the evidence "
12 12 +         "already collected, choose the smallest remaining implementation step, then run one targeted verification."
13 13 +     ),
14 14 }

```

Figure 5 | Code-level differences for the three MiniMax M2.5 Terminal-Bench-2.0 harness modifications retained in the final promoted harness. Red rows denote code removed from the initial harness, and green rows denote the updated harness behavior.

Trace-Level Analysis of Accepted Edits. To examine how accepted harness edits affect agent behavior, we compare representative traces before and after harness modification. Figures 7 and 8 show Terminal-Bench-2.0 examples for MiniMax M2.5 and Qwen3.5-35B-A3B, with the GLM-5 example shown in Appendix Figure 9. Sections B.2 and B.3 provide corresponding SWE-bench Verified and AppWorld cases. Across benchmarks, the retained edits are model- and task-environment-specific and align with the dominant failure mechanisms observed under each initial harness.

For MiniMax M2.5, the accepted edits emphasize early artifact creation and bounded execution. The bootstrap instruction is changed from merely identifying the smallest relevant edit surface to identifying the required output artifact and creating an initial version as early as possible. The runtime policy is also enabled with a limit on total tool messages, encouraging the agent to redirect rather than continue open-ended tool use. Figure 7 shows that this changes the agent from prolonged dataset exploration to a concrete workflow: identifying the relevant metadata split, computing the required count, writing the answer file, and reading it back before stopping.

For Qwen3.5, the promoted harness adds constraints for dependency prechecking, loop breaking, command-retry discipline, and artifact-focused recovery after tool errors. Figure 8 illustrates how these edits change failure recovery behavior. Under the initial harness, the agent creates the required extractor script, encounters overwrite and edit failures, repeatedly tries to modify the same artifact, and ultimately deletes `/app/extract.js` before stopping; the verifier therefore fails because the required file is missing. Under the edited harness, a tool-error-triggered system prompt redirects the agent toward the missing artifact: it recreates the extractor, fixes the parsing logic, writes the output file, performs targeted validation of the JSON result, and leaves the required artifact present for the verifier.

For GLM-5, the accepted edits focus on persistent environment changes and the transition from exploration to implementation. The edited harness instructs the agent to make installed tools or path changes persist across shell sessions and to verify tool accessibility after modifying the environment.

Qwen3.5: dependency precheck

```

1 1 def build_bootstrap_instruction() -> str:
2 2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 3 + return (
4 4 +     "Start by inspecting the workspace and identifying the smallest relevant edit surface. Validate required "
5 5 +     "Python modules before task execution. Explicitly check for missing modules and report them or install them "
6 6 +     "before proceeding."
7 7 + )

```

Qwen3.5: exploration and missing-artifact loop breaker

```

1 1 def build_execution_instruction() -> str:
2 2 - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3 3 + return (
4 4 +     "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task. Do not perform "
5 5 +     "more than 3 consecutive 'explore' or 'execute' steps without making a 'change' (writing code, editing files) "
6 6 +     "or a decisive conclusion. If you are stuck in a loop of exploration, stop and try a different approach. "
7 7 +     "CRITICAL: When you identify a missing required artifact (e.g., FileNotFoundError, explicit dependency check "
8 8 +     "failure, or 'no such file or directory' error), you must create that artifact within 1-2 steps. Do not "
9 9 +     "continue exploring alternatives once the missing file is identified - proceed directly to generation via "
10 10 +     "write_file or edit_file."
11 11 + )

```

Qwen3.5: avoid exact command retries

```

1 1 def build_failure_recovery_instruction() -> str:
2 2 - return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
3 3 + return (
4 4 +     "If a tool call fails, inspect the error and adapt; do not blindly retry the same action. Specifically, if a "
5 5 +     "command fails, do not retry the exact same command with the exact same parameters. You must change the "
6 6 +     "parameters, the target, or the strategy before retrying."
7 7 + )

```

Qwen3.5: tool-error-triggered artifact prompt middleware

```

1 +def build_candidate_middleware():
2 +    """Candidate-provided narrow prompt middleware synthesized from middleware_policy."""
3 +    middleware = _build_prompt_middleware(
4 +        "ArtifactCreationMiddleware",
5 +        lambda: (
6 +            "CRITICAL: A tool call failed due to a missing file or directory. Do not continue exploring or editing "
7 +            "unrelated files. You MUST locate the missing artifact and create it immediately using write_file or "
8 +            "execute build commands. Failure to create the artifact is a terminal failure."
9 +        ),
10 +        predicate=_has_tool_error,
11 +    )
12 +    return (middleware,) if middleware is not None else ()

```

Figure 6 | Code-level differences for the four Qwen3.5-35B-A3B Terminal-Bench-2.0 harness modifications retained in the final promoted harness. Red rows denote code removed from the initial harness, and green rows denote the updated harness behavior.

It also adds a verification-stage constraint: if the agent has been exploring without producing required artifacts, it should transition to implementing and testing a solution. Figure 9 shows this behavior in a build task. The initial harness spends substantial budget on long external downloads and later rationalizes failed sanity checks, whereas the edited harness switches strategy after timeout evidence, validates alternative sources early, repairs the failing render check, and only then finalizes.

Together with the cross-benchmark cases in Sections B.2 and B.3, these examples illustrate concrete workflow changes associated with the quantitative gains in Table 1, including earlier artifact creation, recovery from failed actions, complete state retrieval, and explicit verification before completion. The trace evidence is consistent with the intended mechanism of Self-Harness: recurring execution failures are translated into targeted changes to agent workflow.

5. Conclusion

This paper studied whether a fixed language model can improve the harness that governs its own agent behavior. We introduced Self-Harness, a propose–evaluate–accept framework in which the model is evaluated under the current harness, receives structured evidence from its own execution traces, and proposes bounded edits to declared harness surfaces. Candidate harnesses are then re-evaluated under the same benchmark protocol, and only edits that satisfy a non-regressive acceptance rule are promoted into the harness lineage.

The main lesson is that harness improvement should be treated as an empirical state transition. A

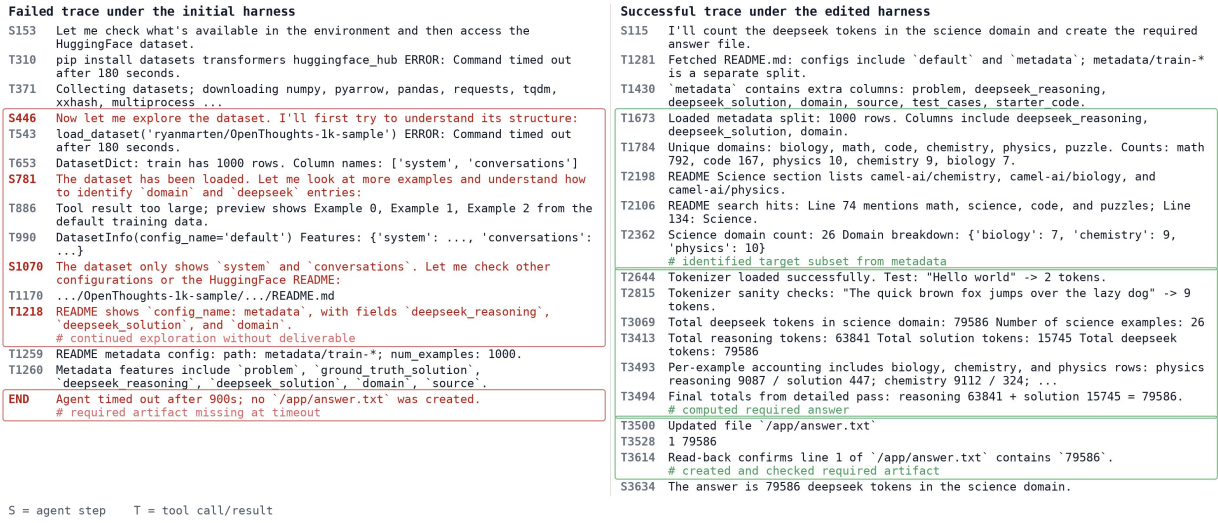


Figure 7 | Representative MiniMax M2.5 traces on the Terminal-Bench-2.0 count-dataset-tokens task. Left: the initial harness ends without the required answer artifact. Right: the edited harness produces and verifies /app/answer.txt.

useful harness edit must specify the behavior it aims to change, the surface it modifies, the evidence that motivates it, and the evaluation result that justifies promotion. By keeping the model, evaluator, and benchmark protocol fixed, Self-Harness isolates whether improved behavior comes from changes to the harness scaffold.

Our experiments across Terminal-Bench-2.0, SWE-bench Verified, and AppWorld instantiate this protocol with a minimal DeepAgent-based baseline harness and three model backends. Self-Harness improves Pass (%) across all nine model-benchmark combinations while preserving held-in and held-out performance under the acceptance rule. The retained edits are small, auditable changes to configurable harness surfaces that address distinct bottlenecks in artifact handling and runtime control, software-patch verification, and application-state retrieval, suggesting that even sparse initial harnesses can support useful self-improvement when proposals are constrained by execution evidence and validated by regression testing.

Self-Harness also has important limits. It studies bounded harness edits under fixed benchmarks, not open-ended self-improvement. Accepted edits may still reflect benchmark-specific failure patterns, and the protocol depends on the quality of verifier outcomes and trace records. Higher-stakes harness changes would require stronger acceptance gates than pass-rate non-regression alone.

More broadly, Self-Harness points toward a style of agent engineering in which harnesses evolve through recorded, testable, and reversible changes. Future work can further explore application of self-harness-style self-improvement in broader environments, but the core requirement remains the same: self-improvement should be grounded in behavioral evidence rather than only in the proposer's rationale for a plausible edit.

References

Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluation of llm agents through iterative tournament competitions. In *International Conference on Machine Learning*. PMLR, 2026.

- GLM-5 Team. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- LangChain. Deepagents, 2026. URL <https://github.com/langchain-ai/deepagents>. Software framework.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Yongyuan Liang, Shijie Zhou, Yu Gu, Hao Tan, Gang Wu, Franck Dernoncourt, Jihyung Kil, Ryan A Rossi, and Ruiyi Zhang. Anticipatory planning for multimodal ai agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5925–5935, 2026.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. Dive into claude code: The design space of today’s and future ai agent systems, 2026. URL <https://arxiv.org/abs/2604.14228>.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, 2021. URL <https://arxiv.org/abs/2107.13586>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan,

Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.

MiniMax. Minimax m2.5: Built for real-world productivity, February 2026. URL <https://www.minimax.io/news/minimax-m25>. Official model report.

Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.

OpenAI. Codex, 2026. URL <https://openai.com/codex/>. Product page.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.

Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025. URL <https://arxiv.org/abs/2505.20286>.

Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>. Official model report and model card for Qwen3.5-35B-A3B.

Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2025. URL <https://arxiv.org/abs/2406.06608>.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, 2024. URL <https://arxiv.org/abs/2310.11324>.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.

- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Bhuwan Dhingra, and Ashish Sabharwal. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076. Association for Computational Linguistics, 2024.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2201.11903>.
- Siyuan Xu, Shiyang Li, Xin Liu, Tianyi Liu, Yixiao Li, Zhan Shi, Zixuan Zhang, Zilong Wang, Qingyu Yin, Jianshu Chen, et al. Controllable and verifiable tool-use data synthesis for agentic reinforcement learning. *arXiv preprint arXiv:2604.09813*, 2026.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL <https://arxiv.org/abs/2310.02304>.
- Hangfan Zhang, Siyuan Xu, Zhimeng Guo, Huaisheng Zhu, Shicheng Liu, Xinrun Wang, Qiaosheng Zhang, Yang Chen, Peng Ye, Lei Bai, et al. The path of self-evolving large language models: Achieving data-efficient learning via intrinsic feedback. *arXiv preprint arXiv:2510.02752*, 2025a.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025b. URL <https://arxiv.org/abs/2505.22954>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2026. URL <https://arxiv.org/abs/2510.04618>. ICLR 2026.
- Shao Zhang, Xihuai Wang, Wenhao Zhang, Chaoran Li, Junru Song, Tingyu Li, Lin Qiu, Xuezhi Cao, Xunliang Cai, Wen Yao, Weinan Zhang, Xinbing Wang, and Ying Wen. Leveraging dual process theory in language agent framework for real-time simultaneous human-AI collaboration.

In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4081–4108, Vienna, Austria, July 2025c. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.206. URL <https://aclanthology.org/2025.acl-long.206/>.

Ningyan Zhu, Huacan Wang, Jie Zhou, Feiyu Chen, Shuo Zhang, Ge Chen, Chen Liu, Jiarou Wu, Wangyi Chen, Xiaofeng Mou, and Yi Xu. Semaclaw: A step towards general-purpose personal ai agents through harness engineering, 2026. URL <https://arxiv.org/abs/2604.11548>.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Juergen Schmidhuber. Language agents as optimizable graphs, 2024. URL <https://arxiv.org/abs/2402.16823>.

A. Benchmark Configurations and Reproducibility Details

A.1. Model Inference Services

MiniMax M2.5 and GLM-5 were accessed through hosted inference services, using MiniMax’s hosted API and OpenRouter, respectively.¹ Qwen3.5-35B-A3B was deployed locally on four NVIDIA H200 GPUs using an internal image derived from the public SGLang Docker image `lmsysorg/sglang:v0.5.12-cu129`.²

A.2. Benchmark Configurations

Shared reproducibility settings. Each candidate evaluation uses two repeated attempts. We keep the task sets, model backends, benchmark environments, evaluators, and all non-editable runtime settings fixed within each comparison.

Terminal-Bench-2.0 configuration. We use Harbor³ as the execution environment for all Terminal-Bench-2.0 tasks. Evaluations use a 2 MB/s outbound network bandwidth cap. To reduce failures caused by incidental network latency rather than agent behavior, we mirror a subset of external resources required by Terminal-Bench-2.0 tasks when the original resources are stable but slow to download. Tasks that depend on external resources that cannot be accessed reliably, as well as tasks requiring multimodal inputs unsupported by the initial harness, are excluded from the main 64-case evaluation set. This configuration keeps the benchmark environment controlled while preserving the core terminal-interaction setting of Terminal-Bench-2.0.

SWE-bench Verified configuration. We use a fixed 100-case subset of SWE-bench Verified, partitioned into 67 held-in and 33 held-out cases. Cases are sampled proportionally by repository, and the resulting split is fixed across all harness variants. Each task is run in an isolated per-instance Docker sandbox initialized with its repository workspace. The resulting patch is scored by the official SWE-bench evaluator in the corresponding prebuilt, task-specific instance image.⁴ We use a 3,600-second case timeout and a 120-second tool timeout.

AppWorld configuration. We use a fixed 180-example AppWorld evaluation set with 90 held-in and 90 held-out examples. The held-in split contains all examples from AppWorld’s official `train` partition, while the held-out split is proportionally sampled from the official `test_normal` and `test_challenge` partitions and then fixed across all harness variants. Each task is executed in a fresh AppWorld context with isolated task state and is scored using AppWorld’s official evaluator. We allow at most 50 agent turns per task.

B. Expanded Experimental Analyses

B.1. Terminal-Bench-2.0

We report the GLM-5 evolution trajectory, retained edits, and a representative Terminal-Bench-2.0 build-task trace.

¹MiniMax token plan: <https://platform.minimax.io/subscribe/token-plan>; OpenRouter GLM-5 endpoint: <https://openrouter.ai/z-ai/glm-5>. Accessed on 2026/05.

²<https://hub.docker.com/r/lmsysorg/sglang/tags?name=v0.5.12-cu129>.

³<https://www.harborframework.com/docs/tutorials/running-terminal-bench>.

⁴Official SWE-bench images are available at <https://hub.docker.com/u/swebench>; per-instance repositories follow the naming pattern `swebench/sweb.eval.x86_64.<instance-id>`.

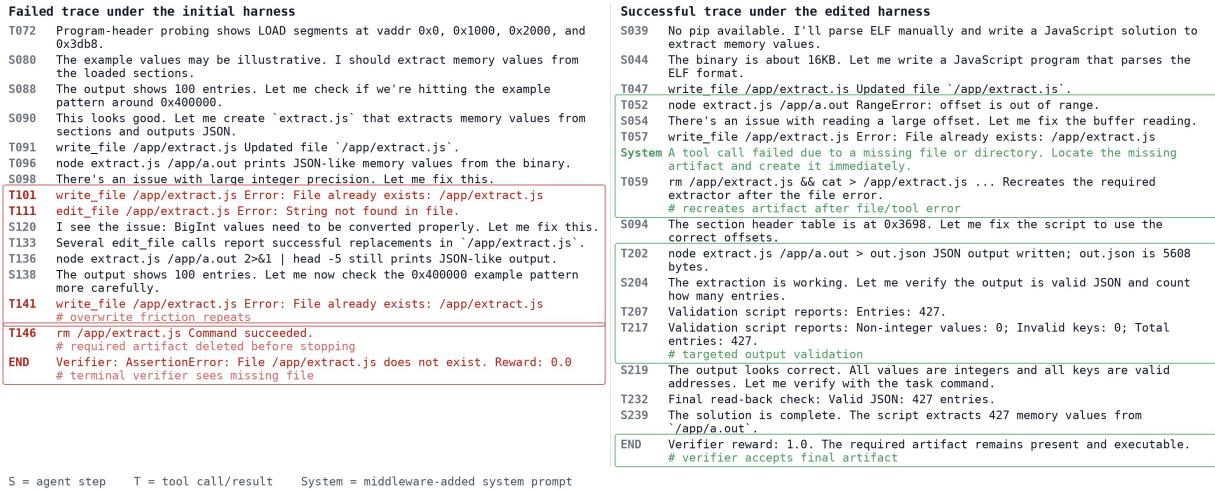


Figure 8 | Representative Qwen3.5 traces on the Terminal-Bench-2.0 `extract-elf` task. Left: the initial harness ends with the required extractor missing. Right: the edited harness restores the extractor and validates its output.

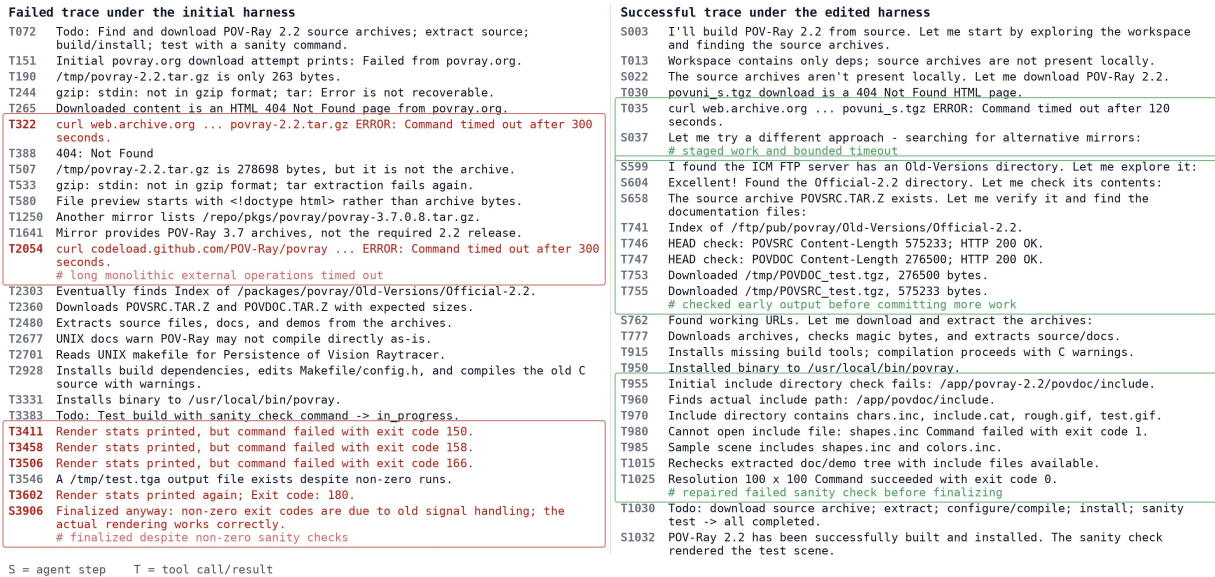


Figure 9 | Case study of a GLM-5 harness edit on the Terminal-Bench-2.0 `build-pov-ray` task. Left: a failed trace under the initial harness, where long monolithic external downloads consume large tool budgets and the agent later finalizes despite repeated non-zero sanity checks. Right: a successful trace under the edited harness, where the agent uses bounded staged operations, checks external archive evidence before committing more work, and repairs the failed sanity check before finalizing.

B.2. SWE-bench Verified

MiniMax M2.5. Overall Pass increases from 46.0% to 52.5%. The promoted harness separates empty-diff detection from targeted local testing: a diff-inspection subagent checks whether a patch exists, and a test-verifier subagent runs the most relevant failing test. In the representative Astropy FITS-card case, the initial harness makes the source change but does not finish verification, whereas the promoted harness resolves dependencies, validates the patch, and completes with the required source change.

Qwen3.5-35B-A3B. Overall Pass increases from 19.5% to 41.5%. The retained harness combines a patch-verifier subagent with a direct pre-submission instruction to inspect the current diff and run targeted tests. In the representative Django window-function case, the initial harness submits a patch without successfully running the relevant test suite, and its sole FAIL-TO-PASS test remains unresolved. The retained harness instead places the SQLite casting guard at the inner window-compatible expression, runs the 49-test expressions_window suite before submission, and passes all 1 FAIL-TO-PASS and 46 PASS-TO-PASS tests.

GLM-5. Overall Pass increases from 52.0% to 55.5%. The promoted harness extends the FAIL-TO-PASS verification cycle with dependency-aware recovery: it parses import failures, identifies and installs missing packages, reruns the exact test, and continues debugging if the test remains unsuccessful. In the Astropy trace, the initial harness submits after only a syntax check, leaving all four FAIL-TO-PASS tests and five PASS-TO-PASS tests failing; the promoted harness completes executable verification, after which all four FAIL-TO-PASS and all 68 PASS-TO-PASS tests pass.

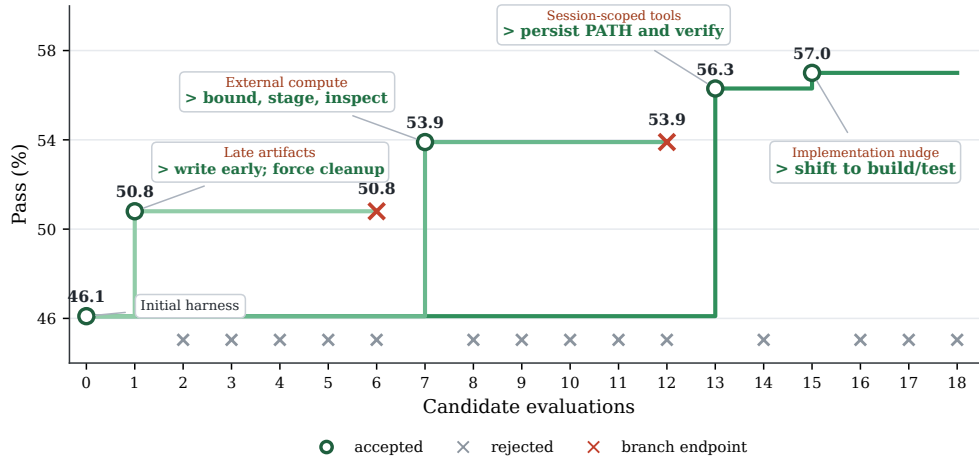
B.3. AppWorld

MiniMax M2.5. Overall Pass increases from 48.6% to 58.9%. The promoted harness combines a state-auditor subagent with pagination and temporal-boundary guidance. In the Venmo case, the initial harness requests payment from all seven participants, whereas the promoted harness exhausts transaction pages, applies the time boundary, identifies four participants who have paid, and requests payment only from the remaining three.

Qwen3.5-35B-A3B. Overall Pass increases from 22.5% to 52.2%. The final harness retains two prompt-level guards: a completion-contract guard distinguishes action-only tasks from information requests, and a pagination guard requires complete record retrieval and a final target-object check. In the Spotify case, the initial harness returns a null play-count result; the promoted harness exhausts the library pages, compares play counts, and returns the expected song.

GLM-5. Overall Pass increases from 44.4% to 85.0%. Its final harness adds an action-versus-information completion rule and exhaustive pagination before state mutation. In the Venmo case, the initial harness uses only the first transaction page and supplies an unnecessary textual answer; the promoted harness exhausts all pages, mutates all 12 records, and completes the action-only task with a null answer.

Across SWE-bench Verified, all three promoted harnesses strengthen verification but choose different structures: MiniMax separates empty-diff and targeted-test checks, Qwen delegates inspection to a patch-verifier subagent, and GLM embeds dependency repair in the verification instruction. Across AppWorld, the runs converge on complete state retrieval and correct completion semantics, again through model-specific harness mechanisms.



(a) Evolution trajectory.

GLM-5: persistent path export

```

1 1 def build_bootstrap_instruction() -> str:
2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 +
4 + "Start by inspecting the workspace and identifying the smallest relevant edit surface. "
5 + "When installing tools or modifying environment variables, ensure changes persist across shell sessions: "
6 + "add executables to system paths such as /usr/local/bin, use /etc/profile.d scripts or ~/.bashrc "
7 + "for PATH modifications instead of session-scoped exports, and verify tool accessibility with "
8 + "'which <tool>' or by executing the tool directly."
9 + )

```

GLM-5: transition from exploration to implementation

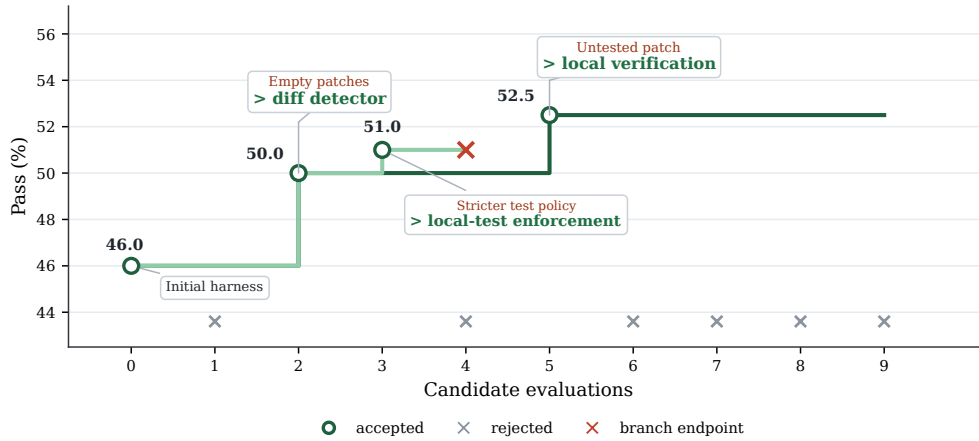
```

1 1 def build_verification_instruction() -> str:
2 - return "Before concluding, verify the result with the most targeted command, file read, or test you can run."
3 +
4 + "Before concluding, verify the result with the most targeted command, file read, or test you can run. "
5 + "If you have been exploring without producing the required artifacts, transition to implementing "
6 + "and testing a solution."
7 + )

```

(b) Retained code-level edits.

Figure 10 | GLM-5 on Terminal-Bench-2.0. Panel (a) shows the evolution from 46.1% to 57.0% overall Pass, with green markers for accepted candidates, gray crosses for rejected candidates, and red crosses for explored branch endpoints. Panel (b) shows the retained edits for persistent environment configuration and the transition from exploration to implementation.



(a) Evolution trajectory.

MiniMax M2.5 SWE: empty-diff detector

```

1 1 def build_subagents() -> list[dict[str, Any]]:
2 2   - return []
3 3   + return [
4 4     + "name": "empty-diff-detector",
5 5     + "description": "Detects when no source changes exist before submission.",
6 6     + "system_prompt": (
7 7       + "You are an empty-diff detector. Do not allow submission of empty patches. "
8 8       + "The submission must contain source code changes."
9 9     + ),
10 10    ]
11 11
12 12 def build_verification_instruction() -> str:
13 13   - return "Run the FAIL_TO_PASS test specified in the problem statement..."
14 14   + return (
15 15     + "Before the final submit call, check if any source code changes have been made. "
16 16     + "If the current diff is empty or no tool calls were made, explore the repository "
17 17     + "and make at least one code edit before submitting.\n\n"
18 18     + "Subagent call policy: call the DeepAgents task tool when empty-diff risk appears."
19 19   )

```

MiniMax M2.5 SWE: targeted local verification before submit

```

1 1 def build_verification_instruction() -> str:
2 2   return (
3 3     + "Before the final submit call, check if any source code changes have been made. ..."
4 4     + "IMPORTANT - Run local verification: After making source edits, run the targeted "
5 5     + "reproduction command or touched-area test visible in this task. Inspect local "
6 6     + "stdout/stderr for test failures, assertion errors, or tracebacks. If the targeted "
7 7     + "output shows a failure, fix the source code and run the test again until it passes.\n\n"
8 8     + "Only call the submit command after local targeted test output passes or shows "
9 9     + "no new failures in the changed code area."
10 10  )

```

(b) Retained code-level edits.

Figure 11 | MiniMax M2.5 on SWE-bench Verified. Overall Pass increases from 46.0% to 52.5%. Panel (a) shows the evolution trajectory, with green markers for accepted candidates and gray crosses for rejected candidates. Panel (b) shows the retained edits, which separate empty-diff detection from targeted local testing.

Failed trace under the initial harness

```

T01  glob: searches for '**/io/fits/card.py'.
T04  grep: searches 'format_float' under '/astropy'.
T06  edit_file: adds an early 'str(value)' branch before the original formatting
      logic.
      # edits early but misses the final required behavior
T08  execute: runs a manual 'Card('HIERARCH ESO IFM CL RADIUS', 0.009125, ...)'
      check.
T10  execute: reruns the same manual check from the default working directory.
T11  execute: 'pip install -e . -q 2>/dev/null || pip install pyerfa -q'.
T12  execute: tests the new '_format_float' logic in an inline Python copy.
T13  execute: compares original and fixed code for exponent edge values.
T14  execute: debugs edge cases for '(1-2**53) * 2 ** exp'.
T15  execute: continues edge-case debugging without converging to benchmark
      behavior.
      # local checks do not converge to a passing fix
T16  execute: 'git diff -- astropy/io/fits/card.py'.
T18  execute: 'echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat patch.txt'.
      # submits despite unresolved targeted behavior
END  Evaluation marks the submitted patch unresolved.
      # evaluator rejects the patch

```

S = agent step T = tool call/result

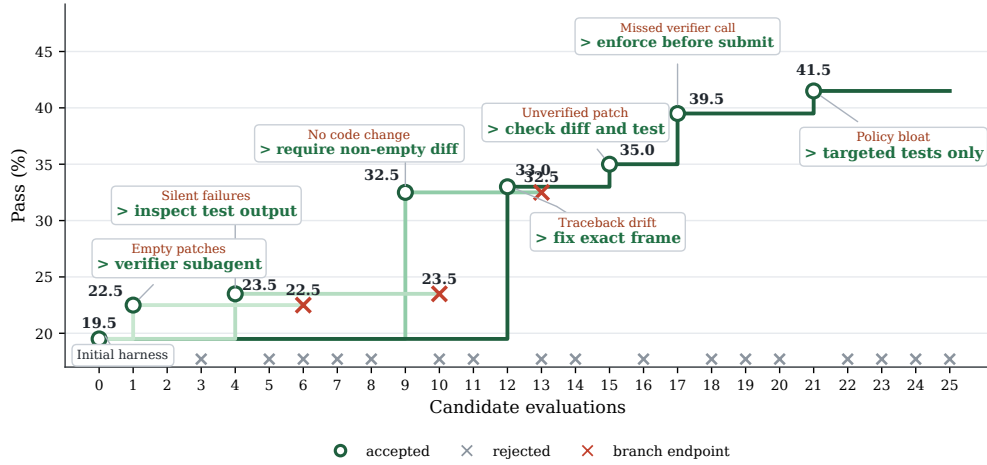
Successful trace under the final SWE harness

```

T07  searches FITS tests and '_format_float' coverage before editing.
T12  execute: compares 'str(value)' and '.16G' for '0.009125' and nearby values.
T17  execute: explicitly checks issue-hint exponent values '[-60, 0, 60]'.
      # targeted edge-case verification before finalizing
T18  edit_file: first changes 'value_str = f"{value:.16G}"' to 'value_str =
      str(value)'.
T25  execute: tests the source function after the first edit.
T29  execute: discovers lowercase 'e' from 'str(1e-10)'.
T31  execute: checks '"E" in s' versus '"e" in s'.
T33  edit_file: adds 'if "e" in value_str: value_str = value_str.replace("e",
      "E")'.
      # fixes the failure discovered by local checks
T36  execute: installs/checks pytest-related test dependencies.
T39  execute: runs comprehensive manual validation of final '_format_float'
      behavior.
T43  execute: final validation over issue and exponent/truncation cases.
      # final targeted validation passes
T45  execute: 'git diff --no-color astropy/io/fits/card.py'.
T48  execute: 'echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat
      /workspace/patch.txt'.
      # submit after diff and relevant checks
END  Final holdout evaluation passes.

```

Figure 12 | SWE-bench Verified Astropy FITS-card float-formatting traces for MiniMax M2.5. Left: the initial harness makes the source change but stops before completing executable verification. Right: the promoted harness resolves the missing dependencies, runs targeted validation, and completes with the required source change.



(a) Evolution trajectory.

Qwen3.5 SWE: check diff and test

```

1  def build_subagents() -> list[dict[str, Any]]:
2  -   return []
3  +   return [
4  +       {
5  +         "name": "patch_verifier",
6  +         "description": "Verify that the current git diff resolves the failing test before submission",
7  +         "system_prompt": (
8  +             "You are a patch verification specialist. "
9  +             "Your task is to inspect the current git diff and run the targeted failing test "
10 +             "to ensure the fix is correct.\n\n"
11 +             "Steps:\n"
12 +             "1. Run 'git diff' to see the changes.\n"
13 +             "2. Identify the failing test mentioned in the problem description or recent test output.\n"
14 +             "3. Run the targeted test command, e.g. 'pytest <test_path>'.\n"
15 +             "4. If the test passes and the diff looks correct, report success.\n"
16 +             "5. If the test fails or the diff is empty/incorrect, report the failure and suggest a fix.\n\n"
17 +             "Do not submit the final output. Only report verification results."
18 +         ),
19 +     },
20 + ]

```

Qwen3.5 SWE: fix exact frame + enforce before submit + targeted tests only

```

1  def build_verification_instruction() -> str:
2  -   return (
3  -       "Run the FAIL_TO_PASS test specified in the problem statement. "
4  -       "If the test fails, parse the failure output, then debug and fix it. "
5  -       "Repeat until the specific FAIL_TO_PASS test passes. "
6  -       "Do not submit while the FAIL_TO_PASS test is still failing."
7  -   )
8  +   return (
9  +       "Before submitting the final output, verify your fix by running the targeted failing test. "
10 +       "Inspect the local test stdout/stderr for assertion failures, tracebacks, or errors. "
11 +       "If the test output shows a failure, do not submit; fix the code and re-run the test. "
12 +       "Only submit when the targeted test output shows no failures and the patch addresses the issue."
13 +   )

```

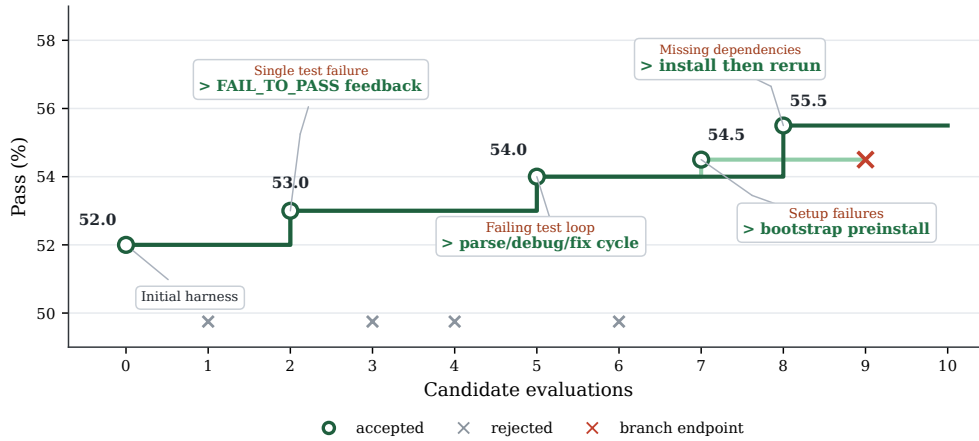
(b) Retained code-level edits.

Figure 13 | Qwen3.5-35B-A3B on SWE-bench Verified. Overall Pass increases from 19.5% to 41.5%. Panel (a) shows the evolution trajectory, and panel (b) shows the retained patch-verifier and pre-submission checks that require inspection of the current diff and execution of targeted tests.

Failed trace under the initial harness	Successful trace under the final SWE harness
<pre> E168 Make 'Window' inherit 'SQLiteNumericMixin'; bypass the cast only at the outer window node. E173 An ad-hoc import check stops at a missing 'asgiref' dependency. E248 Read 'tests/runtests.py', but never execute the relevant window-function suite. # the benchmark-relevant test is never executed E423 Inspect a nonempty source diff in the nested Django repository. E533 Submit the patch without a successful targeted test. # no test evidence gates submission END Official verifier: F2P 0/1; P2P 46/46. SQLite still errors near 'OVER'. # the submitted SQL rewrite remains invalid </pre>	<pre> E038 Guard 'SQLiteNumericMixin': skip the NUMERIC cast for window-compatible expressions. # place the fix where the inner window function is compiled E048 Make 'Window' use the mixin and mark it as window-compatible. E123 A local SQL check confirms that 'LAG(...) OVER (...)' is no longer wrapped by an invalid cast. E168 Run 'python tests/runtests.py expressions_window.tests': 49 tests OK (3 skipped). # targeted official suite passes before submission E183 Inspect the final source-only diff after the targeted suite passes. # inspect the current diff E198 Submit the verified patch. END Official verifier: F2P 1/1; P2P 46/46. Resolved. # all required tests resolve </pre>

E = archived trace event Steps are condensed; verifier counts are exact.

Figure 14 | SWE-bench Verified Django window-function traces for Qwen3.5-35B-A3B. Left: the initial harness submits without running the relevant suite and fails the sole FAIL-TO-PASS test. Right: the retained harness runs that suite before submission and passes all required tests.



(a) Evolution trajectory.

GLM SWE: dependency-aware verification instruction

```

1  def build_verification_instruction() -> str:
2      return (
3          "Run the FAIL_TO_PASS test specified in the problem statement. "
4          "If the test fails, parse the failure output, then debug and fix it. "
5          "Repeat until the specific FAIL_TO_PASS test passes. "
6          "Do not submit while the FAIL_TO_PASS test is still failing."
7      )
8
9  + return (
10     + "Run the FAIL_TO_PASS test specified in the problem statement. "
11     + "Execute the exact test command. "
12     + "If the test fails, parse the specific error. "
13     + "If the failure is a ModuleNotFoundError, ImportError, "
14     + "or missing dependency error, install the missing package "
15     + "using 'pip install <package_name>' and then rerun the test. "
16     + "For missing Python modules, identify the correct package name. "
17     + "After resolving dependency issues, verify the test runs. "
18     + "If the test still fails, debug and fix the code issue. "
19     + "Do not submit while the FAIL_TO_PASS test is still failing "
20     + "or cannot run due to missing dependencies."
21 )

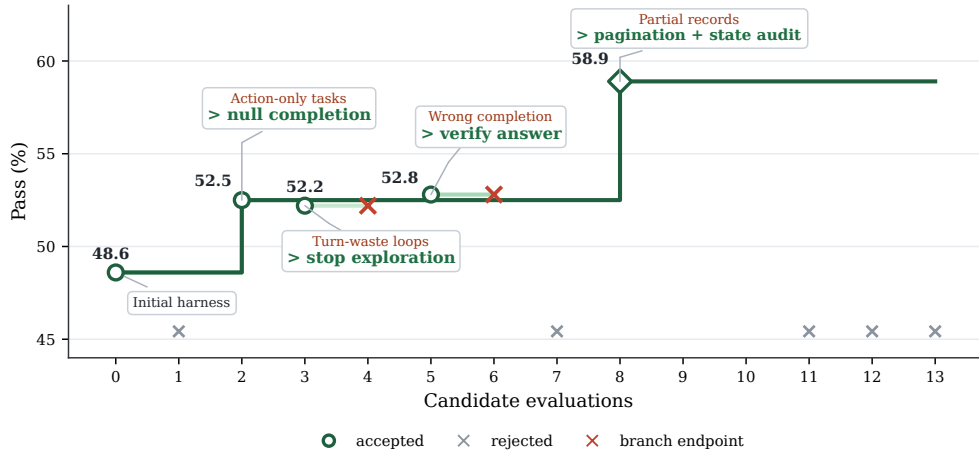
```

(b) Retained code-level edits.

Figure 15 | GLM-5 on SWE-bench Verified. Overall Pass increases from 52.0% to 55.5%. Panel (a) shows the evolution trajectory, and panel (b) shows the retained dependency-aware verification cycle, which recovers from import failures before rerunning the relevant tests.



Figure 16 | SWE-bench Verified Astropy topocentric ITRS transformation traces for GLM-5. Left: the initial harness performs only a syntax check, leaving all four FAIL-TO-PASS tests and five PASS-TO-PASS tests failing. Right: the promoted harness repairs missing dependencies, completes executable verification, and passes all four FAIL-TO-PASS and all 68 PASS-TO-PASS tests.



(a) Evolution trajectory.

MiniMax M2.5 AppWorld: completion-contract state auditor

```

1 def build_subagents() -> list[dict[str, str]]:
2     - return []
3     + return [{
4         "name": "state_auditor",
5         "description": "Verify task completion requirements before complete_task.",
6         "system_prompt": (
7             "If the task asks for information, pass answer with the extracted value. "
8             "If the task only requires an action, pass answer=None or omit answer. "
9             "Use available app APIs to confirm the action before completing."
10        )
11    }]

```

MiniMax M2.5 AppWorld: pagination discipline prompt appendix

```

1 -react_prompt_appendix = ""
2 +react_prompt_appendix = """
3 +Additional AppWorld strategy guidance for this run:
4 +
5 +When using paginated APIs, always check if more pages exist. After retrieving
6 +results, verify you have the complete dataset before filtering or acting on
7 +the data. If an API supports page_index or page_limit, determine whether you
8 +need to iterate through multiple pages to capture all relevant records.
9 +For temporal queries, use proper time boundaries.
10 +"""

```

(b) Retained code-level edits.

Figure 17 | MiniMax M2.5 on AppWorld. Overall Pass increases from 48.6% to 58.9%. Panel (a) shows the evolution trajectory, and panel (b) shows the retained state-auditing, pagination, and temporal-boundary mechanisms used to identify the complete set of target records.

Failed trace under the initial AppWorld harness

```

T01 I need to find the note with individual shares and check Venmo payments.
T03 'search_notes(query='dinner')' returns 401 unauthorized.
T06 logs into simple_note with 'password=<REDACTED>'.
T10 retrieves note: Kayla $20, Sandra $19, Stephen $20, Jamie $15, Anita $23,
    Travis $39, Sabrina $39.
T12 logs into Venmo with 'password=<REDACTED>'.
T14 checks sent payment requests.
T22 checks received payment requests.
T25 'show_transactions(access_token=<REDACTED>, page_index=0, page_limit=20)'.
T26 checks page 1.
T27 checks page 2.
T28 checks page 3.
T29 checks page 4 without finding the relevant paid dinner set.
T30 'none of the dinner participants have paid their shares yet'; decides to
    request all seven.
    # incomplete pagination leads to a wrong payment conclusion
T32 creates requests for Kayla, Sandra, Stephen, Jamie, Anita, Travis, and
    Sabrina.
    # over-requests all 7 users
T39 'apis.supervisor.complete_task()'.
    # finalizes the wrong action set
END Evaluation fails.

```

Successful trace under the final AppWorld harness

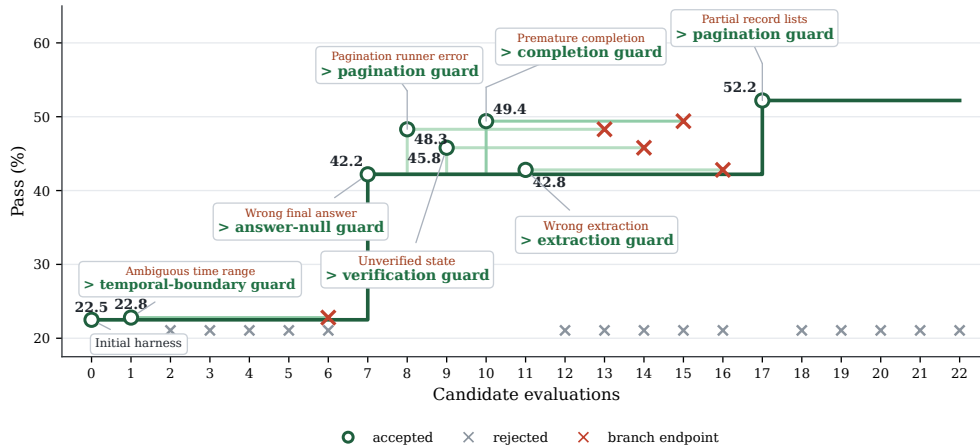
```

T01 I need to find the note with individual shares from the dinner.
T03 'search_notes(query='dinner')' returns 401 unauthorized.
T07 logs into simple_note with 'password=<REDACTED>'.
T11 retrieves the same seven-person share list.
T13 logs into Venmo; output access token redacted.
T15 checks sent payment requests.
T16 checks received payment requests.
T19 'Let me use pagination.' Calls 'show_transactions(..., page_index=1,
    page_limit=20)'.
    # follows pagination discipline from the harness edit
T20 notes temporal boundary: dinner was yesterday, 2023-05-19; checks page 2.
    # applies the relevant temporal boundary
T21 continues transaction pagination through page 3.
T22 continues transaction pagination through page 4.
T23 continues transaction pagination through page 5.
T24 continues transaction pagination through page 6.
T25 already paid: Sabrina $39, Kayla $20, Anita $23, Jamie $15; remaining Sandra
    $19, Stephen $20, Travis $39.
    # verifies complete data before acting
T26 creates exactly three payment requests for Sandra, Stephen, and Travis.
    # requests only the remaining shares
T27 'apis.supervisor.complete_task()'.
END Evaluation passes.

```

S = agent step T = tool call/result

Figure 18 | AppWorld Venmo traces for MiniMax M2.5. Left: the initial harness requests payment from all seven participants without accounting for prior transactions. Right: the promoted harness exhausts the transaction pages, identifies the four participants who have already paid, and requests payment only from the remaining three.



(a) Evolution trajectory.

Qwen3.5 AppWorld: completion-contract prompt guard

```

1 1 def build_react_prompt_template() -> str:
2 2     prompt = BASE_REACT_PROMPT
3 3     prompt += """
4 4 +Completion Contract Check:
5 5 +Before calling complete_task(), decide whether the task asks for an answer.
6 6 +For action-only tasks, pass answer=None or omit answer.
7 7 +For information tasks, pass the exact extracted entity, number, or string.
8 8 +Do not send a summary sentence unless the task explicitly requests one.
9 9 +"""
3 10     return prompt

```

Qwen3.5 AppWorld: pagination prompt guard

```

1 1 def build_react_prompt_template() -> str:
2 2     prompt = BASE_REACT_PROMPT
3 3     prompt += """
4 4 +Critical State Retrieval Rule: Pagination Check.
5 5 +Before filtering, deleting, or mutating list records, inspect pagination fields.
6 6 +If page_index, page_limit, total_count, or has_more appears, iterate all pages.
7 7 +Build decisions from the complete retrieved record set, not from the first page.
8 8 +Re-check the target object after pagination before calling complete_task().
9 9 +"""
3 10     return prompt

```

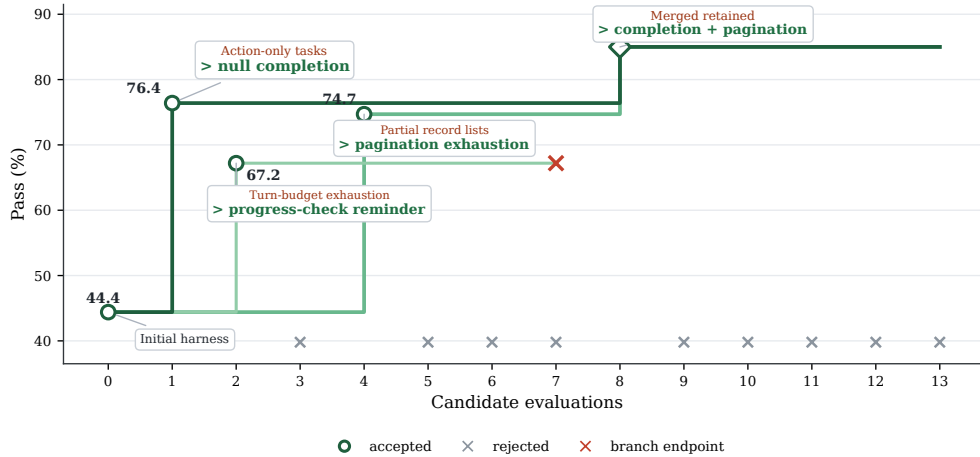
(b) Retained code-level edits.

Figure 19 | Qwen3.5-35B-A3B on AppWorld. Overall Pass increases from 22.5% to 52.2%. Panel (a) shows the evolution trajectory, and panel (b) shows the retained completion-contract and pagination guards, which require complete retrieval and distinguish action-only tasks from information requests.

Failed trace under the initial harness	Successful trace under the edited harness
S01 I need to understand the Spotify APIs for the song library and play counts. T02 'show_song_library' exposes 'page_index' and 'page_limit', default limit 5. S03 The API requires an access_token. I need to get the Spotify password first; try login with email/password variants. T04 Login succeeds with the full email. Fetches the song library and reports 'Total songs fetched: 19'. # acts on an incomplete retrieved set END Evaluation fails: predicted answer is 'null', expected 'distant love'. # wrong completion answer	S01 I need to understand the Spotify APIs to list songs and their play counts. T02 'show_song_library' shows pagination fields: 'page_index', 'page_limit'. S03 'show_song_library' lacks play counts; check 'show_song_privates'. S04 'show_song_privates' does not expose play counts; check 'show_song'. # keeps inspecting before answering T04 'show_song' response includes 'play_count'. S08 The API supports pagination, so I will start with page 0 and iterate through all pages to ensure I get all songs. # follows pagination guard from harness edit T08 Fetch page 0 with 'page_limit=20'; if full, fetch page 1; collect all songs; then call 'show_song' for each song and choose the highest play count. END Evaluation passes: 2/2 tests passed, answers match. # completion contract now matches evaluator

S = agent step T = tool call/result

Figure 20 | AppWorld Spotify traces for Qwen3.5-35B-A3B. Left: the initial harness returns a null play-count result after incomplete retrieval. Right: the promoted harness exhausts the library pages, compares the retrieved play counts, and returns the expected song.



(a) Evolution trajectory.

GLM AppWorld: merged ReAct prompt appendices

```

1 1 def build_react_prompt_template() -> str:
2 2     prompt = BASE_REACT_PROMPT
3 3     prompt += """
4 4 +Important completion rule:
5 5 +If the task asks you to perform actions
6 6 +(create, update, delete, send, follow, like, etc.)
7 7 +without asking a question, call
8 8 +apis.supervisor.complete_task() with no answer argument
9 9 +or answer=None. Do not provide a summary of actions taken.
10 10 +Only pass an answer when the task explicitly asks for information.
11 11 +
12 12 +For APIs that return paginated results
13 13 +(with page_index and page_limit parameters),
14 14 +retrieve ALL pages before processing or mutating state.
15 15 +Increment page_index starting from 0 until an empty list is returned.
16 16 +Do not assume the first page contains all results.
17 17 +Before bulk actions, verify that pagination is exhausted.
18 18 +"""
19 19     return prompt

```

(b) Retained code-level edits.

Figure 21 | GLM-5 on AppWorld. Overall Pass increases from 44.4% to 85.0%. Panel (a) shows the evolution trajectory, and panel (b) shows the retained action-versus-information completion rule and exhaustive-pagination requirement applied before state mutation.

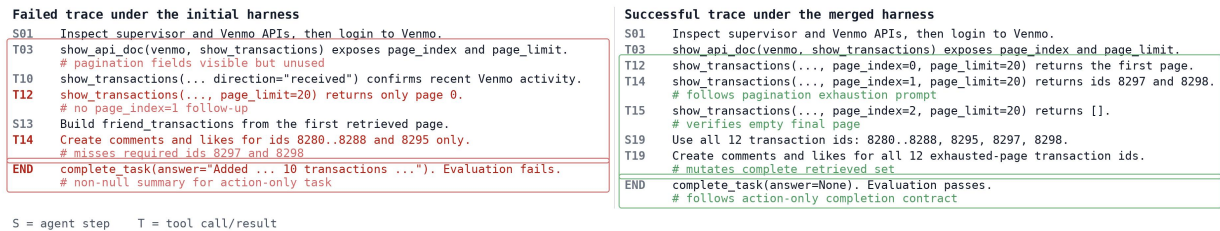


Figure 22 | AppWorld Venmo traces for GLM-5. Left: the initial harness reads only the first transaction page and supplies an unnecessary textual answer. Right: the promoted harness exhausts all pages, mutates all 12 target records, and completes the action-only task with a null answer.